

## 1 Single-Stage vs. Multi-Stage Docker

### Single-Stage Dockerfile:

The single stage Dockerfile built an image with a size of 509MB (with 2.05GB disk usage).

### Multi-Stage Dockerfile:

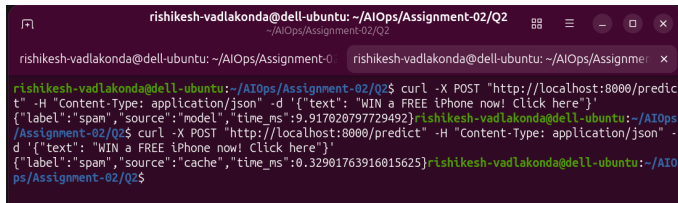
The multi stage Dockerfile built an image with a size of 148MB (with 650MB disk usage) and the percentage reduction is 70.9%.

### Why the multi-stage image is smaller?

The multi stage image is smaller because it left behind the C compiler and the Debian OS which are required to build the python packages in the builder stage.

In the builder stage of my image Debian operating system, system compilers, pip installation caches, and the original requirements.txt file were left behind.

## 2 Multi-Container Orchestration with Docker Compose



```
rishikesh-vadlakonda@dell-ubuntu: ~/AIOps/Assignment-02/Q2
rishikesh-vadlakonda@dell-ubuntu: ~/AIOps/Assignment-02/Q2
rishikesh-vadlakonda@dell-ubuntu:~/AIOps/Assignment-02/Q2$ curl -X POST "http://localhost:8000/predict" -H "Content-Type: application/json" -d '{"text": "WIN a FREE iPhone now! Click here"}'
{"label": "spam", "source": "model", "time_ms": 9.91702877729492}
rishikesh-vadlakonda@dell-ubuntu:~/AIOps/Assignment-02/Q2$ curl -X POST "http://localhost:8000/predict" -H "Content-Type: application/json" -d '{"text": "WIN a FREE iPhone now! Click here"}'
{"label": "spam", "source": "cache", "time_ms": 0.32901763916015625}
rishikesh-vadlakonda@dell-ubuntu:~/AIOps/Assignment-02/Q2$
```

Figure 1: Terminal output demonstrating the API response time difference between a cache miss and a cache hit using Redis.

As you can see in the above figure, the time taken for the first request is 9.91 ms, while the time taken for the same request the next time is 0.33 ms.

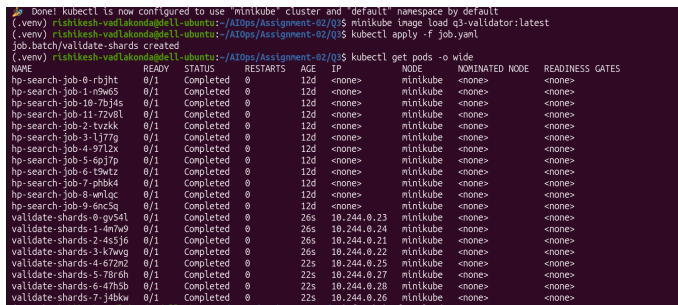
A single Dockerfile can only build and run one isolated container, which makes it useless for setups where an API and a Redis cache need to run together and talk to each other. Docker Compose fixes this spinning up multiple linked containers on a shared network using just one YAML file. However, Compose is limited to running everything on a single host machine, whereas Kubernetes is built to manage and scale containers across a cluster of many different servers.

## 3 Kubernetes Indexed Job: Parallel Data Validation

### Justification for parallelism

Given that the cluster has 2 nodes and 2 allocatable CPU's each. I configured the job with parallelism : 4. To ensure maximum throughput without leaving pods indefinitely pending or overloading the nodes, I set 1 CPU per pod. This allows 4 pods to run parallelly across the clusters. So a total of 8 pods would be completed in 2 waves (4 pods in each).

### Evidence of Parallelism

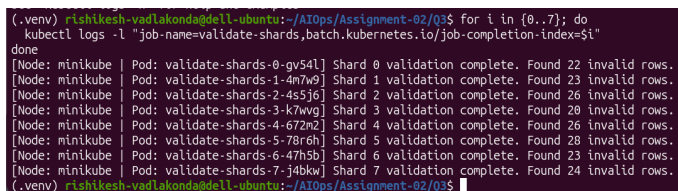


```
Done! kubectl is now configured to use 'minikube' cluster and default namespace by default
(.venv) rishikesh-vadlakonda@dell-ubuntu:~/AIOps/Assignment-02/Q3$ minikube image load g3-validator:latest
(.venv) rishikesh-vadlakonda@dell-ubuntu:~/AIOps/Assignment-02/Q3$ kubectl apply -f job.yaml
job.batch/validate-shards created
(.venv) rishikesh-vadlakonda@dell-ubuntu:~/AIOps/Assignment-02/Q3$ kubectl get pods -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP              NODE               NOMINATED NODE   READINESS GATES
hp-search-job-0-r8jht 0/1     Completed 0           12s   <none>          minikube            <none>            <none>
hp-search-job-1-n9w5s 0/1     Completed 0           12s   <none>          minikube            <none>            <none>
hp-search-job-10-7j4s 0/1     Completed 0           12s   <none>          minikube            <none>            <none>
hp-search-job-11-72v8l 0/1     Completed 0           12s   <none>          minikube            <none>            <none>
hp-search-job-2-tvzkk 0/1     Completed 0           12s   <none>          minikube            <none>            <none>
hp-search-job-3-l777s 0/1     Completed 0           12s   <none>          minikube            <none>            <none>
hp-search-job-4-9712s 0/1     Completed 0           12s   <none>          minikube            <none>            <none>
hp-search-job-5-6gj7p 0/1     Completed 0           12s   <none>          minikube            <none>            <none>
hp-search-job-6-t5wtz 0/1     Completed 0           12s   <none>          minikube            <none>            <none>
hp-search-job-7-pl8k4 0/1     Completed 0           12s   <none>          minikube            <none>            <none>
hp-search-job-8-wl9c 0/1     Completed 0           12s   <none>          minikube            <none>            <none>
hp-search-job-9-6nc5s 0/1     Completed 0           12s   <none>          minikube            <none>            <none>
validate-shards-0-gv54l 0/1     Completed 0           26s   10.244.0.23     minikube            <none>            <none>
validate-shards-1-4m7w9 0/1     Completed 0           26s   10.244.0.24     minikube            <none>            <none>
validate-shards-2-455j6 0/1     Completed 0           26s   10.244.0.21     minikube            <none>            <none>
validate-shards-3-k7wgj 0/1     Completed 0           26s   10.244.0.22     minikube            <none>            <none>
validate-shards-4-672n2 0/1     Completed 0           22s   10.244.0.25     minikube            <none>            <none>
validate-shards-5-78r0h 0/1     Completed 0           22s   10.244.0.27     minikube            <none>            <none>
validate-shards-6-47hsb 0/1     Completed 0           22s   10.244.0.28     minikube            <none>            <none>
validate-shards-7-j4bkw 0/1     Completed 0           22s   10.244.0.26     minikube            <none>            <none>
```

Figure 2: Output of kubectl get pods -o wide confirming the parallelism choice by showing exactly 4 pods running concurrently.

As we can see in the above screenshot, there are 4 pods whose age is 26s (wave-1) and 4 pods whose age is 22s(wave-2).

### Validation Results & Storage Justification



```
(.venv) rishikesh-vadlakonda@dell-ubuntu:~/AIOps/Assignment-02/Q3$ for i in {0..7}; do
  kubectl logs -l "job-name=validate-shards,batch.kubernetes.io/job-completion-index=$i"
done
[Node: minikube] Pod: validate-shards-0-gv54l] Shard 0 validation complete. Found 22 invalid rows.
[Node: minikube] Pod: validate-shards-1-4m7w9] Shard 1 validation complete. Found 23 invalid rows.
[Node: minikube] Pod: validate-shards-2-455j6] Shard 2 validation complete. Found 26 invalid rows.
[Node: minikube] Pod: validate-shards-3-k7wgj] Shard 3 validation complete. Found 20 invalid rows.
[Node: minikube] Pod: validate-shards-4-672n2] Shard 4 validation complete. Found 26 invalid rows.
[Node: minikube] Pod: validate-shards-5-78r0h] Shard 5 validation complete. Found 20 invalid rows.
[Node: minikube] Pod: validate-shards-6-47hsb] Shard 6 validation complete. Found 23 invalid rows.
[Node: minikube] Pod: validate-shards-7-j4bkw] Shard 7 validation complete. Found 24 invalid rows.
(.venv) rishikesh-vadlakonda@dell-ubuntu:~/AIOps/Assignment-02/Q3$
```

Figure 3: Terminal output displaying the invalid row counts extracted from the logs of all 8 completed pods.

To collect the shard validation results, I utilized the Kubernetes API to read the pod logs rather than mounting a shared volume. Extracting results via standard output avoids the limitations of Minikube's default local storage provisioner. In a multi-node setup, a standard ReadWriteOnce volume binds locally to one specific node; if a subsequent pod is scheduled on a different node, it would fail to mount the volume and stall the Job.

### 6-CPU Justification

If there were a total of 6 allocatable CPU's, then I would set parallelism as 6, limiting each pod to a single CPU. So that 6 pods run parallelly, in this case too 8 pods would take 3 waves to complete but the 2nd wave only consists 2 pods.

## 4 Kubernetes Deployments: Self-Healing & Rolling Updates

### Self-Healing

```
rishikesh-vadlakonda@dell-ubuntu: ~/AIOPS/Assignment-02/Q4
valdate-shards-7-j4bkw 0/1 Completed 0 52m
(.venv) rishikesh-vadlakonda@dell-ubuntu:~/AIOPS/Assignment-02/Q4$ kubectl delete pod span-api-deploy-ment-777fb5d9c9-bgsc7
pod "span-api-deployment-777fb5d9c9-bgsc7" deleted from default namespace
(.venv) rishikesh-vadlakonda@dell-ubuntu:~/AIOPS/Assignment-02/Q4$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
hp-search-job-0-rbjht               0/1     Completed 0           12d
hp-search-job-1-n9w65               0/1     Completed 0           12d
hp-search-job-10-7bj4s              0/1     Completed 0           12d
hp-search-job-11-72v8l              0/1     Completed 0           12d
hp-search-job-2-tvzkk               0/1     Completed 0           12d
hp-search-job-3-lj77g               0/1     Completed 0           12d
hp-search-job-4-97L2x               0/1     Completed 0           12d
hp-search-job-5-6pj7p               0/1     Completed 0           12d
hp-search-job-6-49wtz               0/1     Completed 0           12d
hp-search-job-7-phbk4               0/1     Completed 0           12d
hp-search-job-8-wmlqc               0/1     Completed 0           12d
hp-search-job-9-6nc5q               0/1     Completed 0           12d
span-api-deployment-777fb5d9c9-pkk87 1/1     Running   0           8s
span-api-deployment-777fb5d9c9-wzp6c 1/1     Running   0           51s
valdate-shards-0-gv54l              0/1     Completed 0           53m
valdate-shards-1-4m7w9              0/1     Completed 0           53m
valdate-shards-2-4s5j6              0/1     Completed 0           53m
valdate-shards-3-k7wvg              0/1     Completed 0           53m
valdate-shards-4-672m2              0/1     Completed 0           53m
valdate-shards-5-78r6h              0/1     Completed 0           53m
valdate-shards-6-47h5b              0/1     Completed 0           53m
valdate-shards-7-j4bkw              0/1     Completed 0           53m
```

Figure 4: Terminal output demonstrating the ReplicaSet immediately creating a new pod after manual deletion.

### Rolling Update

```
rishikesh-vadlakonda@dell-ubuntu: ~/AIOPS/Assignment-02/Q4
=> CACHED [stage-1 2/5] WORKDIR /app 0.0s
=> CACHED [builder 2/4] WORKDIR /build 0.0s
=> CACHED [builder 3/4] COPY requirements.txt 0.0s
=> CACHED [builder 4/4] RUN pip install --no-cache-dir --target=/build/deps -r requirements.t 0.0s
=> CACHED [stage-1 3/5] COPY --from=builder /build/deps /usr/local/lib/python3.10/site-packag 0.0s
=> [stage-1 4/5] COPY api.py . 0.0s
=> [stage-1 5/5] COPY spam_model.joblib . 0.0s
=> exporting to image 0.1s
=> exporting layers 0.0s
=> exporting manifest sha256:22f85a472f4df71940e3d9cbe83d4e916ada534563fd9b968cbcfcbdf1b3 0.0s
=> exporting config sha256:892474fde151a0eb5085c1b2f2b78d12828816ebf2eabbab4b7ef3cfe72e8b 0.0s
=> exporting attestation manifest sha256:524a0b03cc5befd5753b040911ea26d5850f1b86b96dba3c0 0.0s
=> exporting manifest list sha256:f15fe4b664c5d5650cebf4248f8b6a249970bc9570bbc4bcf895b676 0.0s
=> naming to docker.io/library/span-api:v2 0.0s
=> unpacking to docker.io/library/span-api:v2 0.0s
(.venv) rishikesh-vadlakonda@dell-ubuntu:~/AIOPS/Assignment-02/Q4$ minikube image load span-api:v2
(.venv) rishikesh-vadlakonda@dell-ubuntu:~/AIOPS/Assignment-02/Q4$ kubectl set image deployment/span-api-deployment api=span-api:v2
deployment.apps/span-api-deployment image updated
(.venv) rishikesh-vadlakonda@dell-ubuntu:~/AIOPS/Assignment-02/Q4$ kubectl rollout status deployment/span-api-deployment
Waiting for deployment "span-api-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "span-api-deployment" rollout to finish: 1 old replicas are pending termination...
deployment "span-api-deployment" successfully rolled out
(.venv) rishikesh-vadlakonda@dell-ubuntu:~/AIOPS/Assignment-02/Q4$ kubectl rollout history deployment/span-api-deployment
deployment.apps/span-api-deployment
REVISION    CHANGE-CAUSE
1           <none>
2           <none>
```

Figure 5: Rollout status and history confirming the successful zero-downtime update to the v2 image.

### Job vs Deployment

**Deployments (Serving Workloads):** Deployments are the correct architectural fit for continuous serving workloads (such as a REST API) because their self-healing and rolling-update capabilities guarantee high availability, zero-downtime deployments, and automatic recovery from infrastructure failures.

**Jobs (Batch Workloads):** Jobs are the correct fit for finite batch workloads (such as the CSV data validation in Question 3) because they enforce strict run-to-completion semantics, ensuring that pods process their assigned shards, report their outputs, and automatically terminate upon completion without requiring infinite uptime.

The controller responsible for this is the ReplicaSet controller, it continuously monitors the current status of the pods and the desired status specified in the Deployment manifest. When a running pod is manually deleted, the actual replica count temporarily drops to 1. So the ReplicaSet controller schedules the creation of a new replacement pod to restore the desired state back to 2.

If this API application had been deployed as a Job instead of a Deployment, updating the container image version in the manifest would not have triggered a rolling update of live, running pods.